

Portal Trade Reconstruction Validation

Technical audit generated from the current Portal workspace.

1. Executive Summary

Portal's current reconstruction layer is **partially trustworthy**.

It is reasonably trustworthy for reconstructing simple net-position trade lifecycles from Hyperliquid fills when the selected range contains the full lifecycle and *startPosition* is present. The engine correctly intends to group scale-ins and scale-outs into one reconstructed position lifecycle rather than turning every fill into a fake trade. That is the right model for behavioral diagnosis.

It is **not trustworthy enough yet** for diagnosis-grade session intelligence, overnight PnL, day-by-day performance, fee-net profitability, or time-window recommendations. Sessions are currently thin 90-minute gap groups, overnight is only a UTC bucket in baseline metrics, and Portal does not persist a user-selected timezone for imported wallet history. Net PnL is also not independently reconciled against fees and funding; it uses Hyperliquid *closedPnl* plus attributed funding.

Validation scan generated during this audit:

- Debug output: `.portal/audits/trade-reconstruction-latest.json`
- Wallet: `0x36fF15F480BDbAF92B868B54BB0B9484e82E4b8a`
- Environment: `mainnet`
- Range: `2026-04-23T00:00:00.000Z` to `2026-05-22T23:59:59.999Z`
- Counts: 57 fills, 2000 orders, 206 funding rows, 9 ledger rows, 28 reconstructed trades, 26 closed trades, 2 open trades, 9 sessions, 1 boundary fill excluded, 1 unmatched fill order ID, 0 duplicate fills detected, 0 invalid timestamps detected.

Official Hyperliquid references used:

- Info endpoint: <https://hyperliquid.gitbook.io/Hyperliquid-docs/for-developers/api/info-endpoint>
- Perpetuals info endpoint: <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/info-endpoint/perpetuals>
- Rate limits and user limits: <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/rate-limits-and-user-limits>

2. Data Intake Map

Primary historical import path:

1. UI collects source, environment, wallet address, and date range in `src/components/layout/app-shell.tsx`.
2. `getBaselineImportRequestFromDraft` builds a `BaselineImportRequest`.
3. `runHistoricalBaselineScan` validates the address and calls `runBaselineImportFetch`.
4. `runBaselineImportFetch` calls the Hyperliquid info endpoint for fills, orders, funding, non-funding ledger updates, and clearinghouse state.
5. Normalizers convert wire rows into `ImportedFill`, `ImportedOrder`, `ImportedFundingRecord`, and `ImportedLedgerUpdate`.
6. `reconstructImportedTrades` groups fills into `ImportedTrade` records and `ImportedSession` records.
7. `buildBaselineProfile` turns closed trades into summary metrics, behavior buckets, and warnings.
8. App shell stores the result in `localStorage` under `portal.baseline-import`.
9. AI Coach and Operator Diagnosis can consume the baseline as external context.

Hyperliquid data fetched by `src/features/baseline-import/hyperliquid-import-client.ts`:

Source	Portal Function	Endpoint / Payload	Date Range	Failure Behavior	Known Limit
User fills	<i>fetchUserFillsByTime</i>	<i>POST /info, { type: "userFillsByTime", user, startTime, endTime, aggregateByTime: true }</i>	Uses selected start/end in milliseconds	Fatal for scan if this call fails	Official docs: at most 2000 fills per response; only 10000 most recent fills are available
Historical orders	<i>fetchHistoricalOrders</i>	<i>POST /info, { type: "historicalOrders", user }</i>	No date range in request	Soft-fails to []	Official docs: at most 2000 most recent historical orders
User funding	<i>fetchUserFunding</i>	<i>POST /info, { type: "userFunding", user, startTime, endTime }</i>	Uses selected start/end in milliseconds	Soft-fails to []	Response completeness depends on API availability and range
Non-funding ledger	<i>fetchUserNonFundingLedgerUpdates</i>	<i>POST /info, { type: "userNonFundingLedgerUpdates", user, startTime, endTime }</i>	Uses selected start/end in milliseconds	Soft-fails to []	Used only for context totals, not trade lifecycle reconstruction
Account snapshot	<i>fetchClearinghouseState</i>	<i>POST /info, { type: "clearinghouseState", user }</i>	Current snapshot only	Soft-fails to null	Not used to repair open/boundary positions

Mainnet/testnet behavior:

- *mainnet* uses <https://api.hyperliquid.xyz/info>.
- *testnet* uses <https://api.hyperliquid-testnet.xyz/info>.
- The Historical Baseline wizard defaults to *testnet*.

Wallet handling:

- Address validation requires a 42-character 0x EVM-style address.
- Requests preserve trimmed user casing.
- Stored bundle IDs lowercase the address.
- There is no user-role check yet for user vs agent vs vault vs subaccount, despite Hyperliquid exposing user role information.

3. Trade Lifecycle Reconstruction

src/features/baseline-import/reconstruct-trades.ts reconstructs trades from fills using one active net-position lifecycle per coin.

Core algorithm:

1. Sort fills by coin, time, order ID, and trade ID.
2. Track one active mutable trade per coin.
3. Use *fill.startPosition* when present; otherwise fall back to current active position or zero.
4. Compute *after = startPosition + signedSize*.
5. If no active trade exists and *startPosition* is non-zero, count the fill as a boundary fill and exclude it, unless it reverses into a new in-range position.
6. If position size increases in the same direction, add the fill as a scale-in.
7. If position size decreases but stays same direction, add the fill as a reduce/scale-out.
8. If position reaches zero, close the trade.
9. If the sign flips, split the fill into a close fragment and an open fragment.
10. Attach matching historical orders by *orderId*.
11. Derive execution mode from non-reduce, non-position-TP/SL matched orders.

12. Derive entry-style proxy from execution mode.
13. Attach funding rows by coin and time window.
14. Group trades into sessions with a 90-minute inactivity gap.

This design intentionally avoids treating each fill as a separate behavioral trade. It is directionally correct for Portal's goal: a "trade" is a useful position lifecycle, not an exchange fill row.

4. Trade Record Schema

Current *ImportedTrade* contains:

- *coin*
- *direction*
- *entryTime*
- *endTime*
- *durationMinutes*
- *entryPrice*
- *exitPrice*
- *maxSize*
- *openFills*
- *addFills*
- *reduceFills*
- *closeFills*
- *orderIds*
- *rawOrderMatches*
- *executionClosedPnl*
- *fundingPnl*
- *grossFees*
- *netPnl*
- *status*
- *executionMode*
- *entryStyleProxy*
- *reconstructionConfidence*
- *sessionId*

Diagnosis-ready trade records should add:

- *sourceCompleteness*: complete / boundary-open / boundary-close / possibly-truncated / order-enrichment-missing
- Explicit *grossPnlBeforeFees*
- Explicit *feePnl* or *feesPaid*
- Explicit *netPnlAfterFeesAndFunding*
- Explicit fee semantics: whether *closedPnl* is fee-inclusive or fee-exclusive

- *notionalEntryValue*, *notionalExitValue*, and *maxNotionalExposure*
- *scaleInCount*, *scaleOutCount*
- *reversalParentId* / *reversalChildId*
- *openedBeforeRange*, *closedAfterRange*
- *currentOpenSnapshot* for open trades
- *fundingAttributionConfidence*
- *orderMatchCompleteness*
- *duplicateFillCount*
- *invalidTimestampSourceCount*
- *timezoneContext*
- *diagnosisConfidence*

5. Current Trade Reconstruction Weaknesses

Major weaknesses:

- **Historical orders are not date-ranged.** Portal fetches the latest 2000 historical orders and matches by *orderId*; older fills in the selected window can lose order context.
- **Order cap can silently degrade classifications.** The scan correctly warns at 2000 orders, but still shows market/limit/hunted/chased summaries for whatever matched.
- **Boundary positions are excluded, not repaired.** Closes/reduces from positions opened before the range are counted as boundary fills and removed from recommendation-grade trades.
- **Open positions at range end are incomplete.** Open trades keep entry state and fees, but no mark-to-market unrealized PnL is included.
- ***parseTimestamp* falls back to *Date.now()*.** Invalid source timestamps become current-time rows instead of explicit invalid rows. The new debug audit detects invalid raw timestamps where possible, but production normalization still hides them.
- **No explicit duplicate detection in the reconstruction path.** The new local debug output counts duplicates, but the scan still does not remove or warn on duplicates in product state.
- **Fee math is not independently reconciled.** $netPnl = executionClosedPnl + fundingPnl$; fees are stored separately and $grossPnlBeforeFees = netPnl + closedGrossFees$. This is only correct if Hyperliquid *closedPnl* is fee-net in the expected way.
- **Funding attribution is approximate.** Funding rows are assigned to trades by same coin and time between entry and close. This can misattribute funding when positions overlap across range boundaries or when position lifecycle is incomplete.
- **Same-coin overlapping independent positions are impossible on a net-position exchange.** The one-active-trade-per-coin model is correct for net positions, but it cannot preserve strategy-level subpositions if a trader mentally separates them.
- **TP/SL and reduce-only order semantics are only partially used.** They are filtered out for entry-mode classification, but not modeled as exit reason, risk management behavior, or protection behavior.
- **HIP-3 / dex-specific coin naming is not modeled in the historical import request.** The scan uses *coin* strings directly and no *dex* parameter.

6. Session Reconstruction

Current sessions are built in *buildSessions* inside *reconstruct-trades.ts*.

Current behavior:

- Sort reconstructed trades by *entryTime*.
- Start a new session when the gap from the previous trade's *endTime* or *entryTime* exceeds 90 minutes.
- Store only: start, end, dominant asset, trade count, trade IDs, net PnL, win count, loss count, notes.

This is **not enough** for Portal's future Operator Diagnosis goal.

It can answer:

- How many grouped sessions exist.
- Which session has the most trades, indirectly through profile metrics.
- Whether any session has more than five trades, through *sessionsOverFiveTrades*.
- How many short-gap trades exist globally.

It cannot reliably answer:

- "From 8:00 PM to 5:00 AM this wallet took 7 trades, lost \$933 net, paid \$91 in fees..."
- Session gross PnL, fees, funding, net PnL after fees/funding.
- Session best/worst trade.
- Session average hold time.
- Session total time in market.
- Session largest notional risk.
- Session direction mix.
- Session asset mix beyond dominant asset.
- Session first trade result.
- Post-loss re-entry count inside the session.
- Weekend vs weekday.
- Overnight by user-local time.
- Session confidence level.

7. Session Record Schema

A diagnosis-ready session record should contain:

- *id*
- *accountAddress*
- *environment*
- *timezone*
- *startedAt*
- *endedAt*
- *sessionType*: same-day / overnight / high-density / post-loss-cluster / weekend / manual
- *tradeIds*
- *totalTrades*
- *closedTrades*

- *openTrades*
- *wins*
- *losses*
- *winRate*
- *grossPnlBeforeFees*
- *fees*
- *funding*
- *netPnlAfterFeesAndFunding*
- *bestTradeId*
- *worstTradeId*
- *averageHoldMinutes*
- *totalTimeInMarketMinutes*
- *largestBaseSize*
- *largestNotionalExposure*
- *assetMix*
- *directionMix*
- *firstTradeResult*
- *postLossReentryCount*
- *shortGapTradeCount*
- *timeBetweenTrades*
- *isHighDensity*
- *isOvernight*
- *isWeekend*
- *boundaryTradeCount*
- *openAtSessionEnd*
- *orderMatchCompleteness*
- *confidence*
- *confidenceReasons*

8. Overnight PnL Validation

Recommended default overnight definition:

- User-local overnight window: 8:00 PM to 5:00 AM.

Current Portal behavior:

- Baseline time-of-day buckets use *new Date(time).getUTCHours()*.
- Baseline "overnight" means *00:00-05:59 UTC*, not user-local *20:00-05:00*.
- Operator Diagnosis time-window buckets use *new Date(timestampMs).getHours()*, which means the runtime/browser local timezone.

- Discipline blocked-time-window rules can accept a timezone, but imported baseline scans do not capture one.
- App-shell date selection uses UTC day boundaries (YYYY-MM-DDT00:00:00.000Z to 23:59:59.999Z).

Verdict:

Portal **cannot currently calculate trustworthy overnight PnL** for the intended user-local 8:00 PM to 5:00 AM definition. The audit debug JSON includes an audit-only UTC overnight counter to expose the gap, but it is not product behavior and should not be used as diagnosis truth.

Required before trusting overnight diagnosis:

- Persist a timezone with the baseline request.
- Bucket trades/sessions in that timezone.
- Define whether a trade belongs to overnight by entry time, exit time, majority duration, or any overlap.
- Aggregate overnight by session, not only by individual trade entry.
- Store overnight confidence and show insufficient evidence when boundary/open/truncated data affects the window.

9. Timezone / Time Bucket Audit

Time handling by layer:

Layer	Current Time Representation	Bucket Timezone	Risk
Hyperliquid fills/orders/funding	Milliseconds from API, normalized to number and ISO	Exchange timestamps are treated as epoch ms	Mostly safe if source timestamp is valid
Normalization	<i>parseTimestamp</i> returns ms, but falls back to <i>Date.now()</i>	N/A	Invalid timestamps can become false current-time rows
Trade reconstruction	Uses millisecond arithmetic	N/A	Duration is safe if timestamps are valid
Baseline date range	UTC day boundaries from date input	UTC	User-selected calendar days may not match user's local trading day
Baseline time buckets	<i>getUTCHours()</i>	UTC	Overnight / cash / evening can be wrong for US users
AI Coach baseline cards	Uses baseline buckets	UTC for baseline	Can repeat weak time claims with wrong timezone
Operator Diagnosis	<i>getHours()</i>	Runtime local	Inconsistent with baseline UTC
Realtime warnings	<i>getHours()</i>	Runtime local	May warn on different hour than imported diagnosis source
Session Log display	<i>Intl.DateTimeFormat</i> without timezone	Runtime local	Display can differ from stored bucket logic

Answers:

- Are timestamps in milliseconds? Mostly yes for imported records and reconstructed trades.
- Are buckets UTC or local? Baseline buckets are UTC; diagnosis buckets are runtime local.
- Does the app currently know the user's timezone? Not for historical baseline scans.
- Could overnight/session conclusions be wrong due to timezone mismatch? Yes, materially.
- What must be fixed before time-based diagnosis? Store timezone, use one bucket helper across baseline/diagnosis/warnings, and annotate confidence.

10. PnL / Fee / Funding Audit

Current PnL fields:

- *executionClosedPnl*: sum of fill *closedPnl* on reduce/close fills.
- *grossFees*: sum of fill fees across open/add/reduce/close fills.
- *fundingPnl*: sum of funding *delta* rows for same coin between trade entry and close.
- *netPnl*: *executionClosedPnl* + *fundingPnl*.
- *grossPnlBeforeFees*: *netPnl* + *closedGrossFees* at profile level.

Reliability:

- Trade win/loss and direction are directionally useful when complete.
- Fee totals are collected, but not proven to be additive to *closedPnl* in the way the UI implies.
- Funding is separately collected, but attribution is approximate.
- Ledger net flow is stored as context, not part of trade PnL.
- Open trade fees are separated from closed headline fees in the profile, which is good.

Audit scan totals from *.portal/audits/trade-reconstruction-latest.json*:

- Total gross PnL before fees: *-\$191.89*
- Total fees: *\$70.18*
- Total funding: *\$0.00*
- Total net PnL: *-\$261.63*

The fee and net relationship should be treated as **medium/low confidence** until closed PnL semantics are explicitly documented in code and reconciled on a known wallet sample.

11. Diagnosis-Readiness Matrix

Insight Type	Current Reliability	Required Data	Current Weakness	Recommendation
Trade count	Ready for diagnosis when no cap/boundary warnings	Complete reconstructed trades	Boundary/open/truncated range can undercount	Show confidence and excluded counts
Win rate	Directional only	Complete closed trades	Boundary trades excluded; open trades ignored	Label as closed-trade win rate
Net PnL	Directional only	Fee/funding-reconciled trade PnL	Fee semantics uncertain	Rename or reconcile before hard claims
Time-of-day performance	Unsafe / not enough evidence	Timezone-aware buckets	Baseline UTC vs diagnosis local mismatch	Must fix before diagnosis redesign
Overnight performance	Unsafe / not enough evidence	User-local overnight sessions	No user timezone; no session-level overnight PnL	Build first-class overnight session records
High-density sessions	Directional only	Rich session records	Current threshold is sessions over five trades after 90-minute grouping	Add session schema and configurable threshold
Post-loss re-entry	Directional only	Close time, next entry, loss result	Current diagnosis allows one-day next trade; baseline has short-gap count separately	Use explicit re-entry windows by session
Size-after-loss	Directional only	Comparable exposure/risk	Uses base size, not notional/leverage/risk	Use notional and risk-adjusted size
Market/limit entry style	Directional only	Complete historical orders	Historical order cap and unmatched orders degrade matching	Add order completeness confidence
Chased/hunted proxy	Unsafe as hard truth	Market context and order intent	Limit = hunted and market = chased is a weak proxy	Keep as labeled proxy only
Asset performance	Ready / directional depending sample	Complete closed trade PnL by asset	Boundary and sample-size issues	Show sample reliability

Insight Type	Current Reliability	Required Data	Current Weakness	Recommendation
Long/short bias	Ready / directional depending sample	Complete closed trades with direction	Net-position direction works, but boundary exclusions matter	Show confidence
Hold-time buckets	Directional only	Complete entry/exit timestamps	Open and boundary trades omitted; bucket sample often thin	Require sample threshold
Day-by-day performance	Unsafe / not enough evidence	Timezone-aware trading day grouping	Diagnosis uses UTC ISO day in overtrading finding	Add local trading-day keys
Final net after fees/funding	Directional only	Closed PnL, fees, funding semantics	Fee semantics not proven	Reconcile against known Hyperliquid account totals

12. Recommended Fixes

Must fix before diagnosis redesign

1. Add a first-class *ReconstructedTrade* confidence model with reasons: complete, boundary-open, open-at-range-end, order-cap-hit, fill-cap-hit, unmatched-orders, invalid-timestamps.
2. Persist timezone in baseline requests and use one shared time-bucket helper across baseline, AI Coach, Operator Diagnosis, warnings, and Discipline activation.
3. Build first-class session records with gross PnL, fees, funding, net PnL, trade count, best/worst trade, high-density, post-loss re-entry, and overnight flags.
4. Clarify fee semantics by verifying whether Hyperliquid *closedPnl* is fee-inclusive for the rows Portal uses; then rename fields accordingly.
5. Replace *parseTimestamp* fallback-to-now with explicit invalid timestamp handling.

Should fix soon

1. Paginate or chunk *userFillsByTime* where possible and flag the official 10000-fill availability limit.
2. Improve historical order enrichment so order matching is range-aware or confidence-aware when the 2000 order cap is hit.
3. Add duplicate fill detection into scan warnings, not only local debug output.
4. Use current *clearinghouseState* to annotate open positions at scan end where possible.
5. Separate TP/SL and reduce-only exit behavior from entry style so protection behavior can feed Operator Diagnosis later.

Nice to have later

1. Add user-role/subaccount/vault validation before scan.
2. Add dex / HIP-3 market context if Portal needs builder-deployed perps.
3. Add chart-context enrichment for chased/hunted beyond market vs limit order type.
4. Preserve raw payload snapshots behind an explicit local audit mode with redaction.
5. Add session-level visual explainers after the engine is trustworthy.

13. Files Reviewed

- *package.json*
- *scripts/write-insight-baseline-audit.ts*
- *scripts/write-trade-reconstruction-audit.ts*
- *src/app/api/account/route.ts*

- `src/components/layout/app-shell.tsx`
- `src/features/account-activity/lib/account-activity-ledger.ts`
- `src/features/ai-coach/lib/ai-warning-aggregation.ts`
- `src/features/ai-coach/lib/evidence-source-mix.ts`
- `src/features/ai-coach/lib/evidence-source-mix.test.ts`
- `src/features/ai-coach/lib/observational-coach.ts`
- `src/features/ai-coach/lib/observational-coach.test.ts`
- `src/features/baseline-import/baseline-import.test.ts`
- `src/features/baseline-import/build-baseline-profile.ts`
- `src/features/baseline-import/debug-output.ts`
- `src/features/baseline-import/hyperliquid-import-client.ts`
- `src/features/baseline-import/index.ts`
- `src/features/baseline-import/normalize-import.ts`
- `src/features/baseline-import/reconstruct-trades.ts`
- `src/features/baseline-import/storage.ts`
- `src/features/baseline-import/types.ts`
- `src/features/diagnosis/lib/diagnosis-realtime-warnings.ts`
- `src/features/diagnosis/lib/diagnosis-realtime-warnings.test.ts`
- `src/features/diagnosis/lib/discipline-plan-activation.ts`
- `src/features/diagnosis/lib/operator-diagnosis.ts`
- `src/features/diagnosis/lib/operator-diagnosis.test.ts`
- `src/features/discipline/lib/discipline-time-window.ts`
- `src/features/journal/lib/journal-insights.ts`
- `src/features/journal/lib/journal-insights.test.ts`
- `src/features/journal/lib/journal-memory-events.ts`
- `src/features/journal/lib/journal-memory-insights.ts`
- `src/features/session-log/lib/session-log.ts`
- `.portal/audits/trade-reconstruction-latest.json`

14. Test Results

- `npm test -- src/features/baseline-import/baseline-import.test.ts`: passed, 23 tests.
- `npm test -- src/features/baseline-import/baseline-import.test.ts src/features/ai-coach/lib/observational-coach.test.ts src/features/ai-coach/lib/ai-warning-aggregation.test.ts src/features/ai-coach/lib/evidence-source-mix.test.ts src/features/diagnosis/lib/operator-diagnosis.test.ts src/features/diagnosis/lib/diagnosis-realtime-warnings.test.ts src/features/journal/lib/journal-insights.test.ts src/features/journal/lib/journal-memory-insights.test.ts src/features/session-log/lib/session-log.test.ts src/features/account-activity/lib/account-activity-ledger.test.ts`: passed, 113 tests.
- `npm run audit:trade-reconstruction -- --address 0x36f15f480BDbAF92B868B54BB0B9484e82E4b8a --environment mainnet --start 2026-04-23 --end 2026-05-22`: passed and wrote `.portal/audits/trade-reconstruction-latest.json`.

- *npm test*: passed, 629 tests.
- *npx tsc --noEmit --pretty false*: passed.